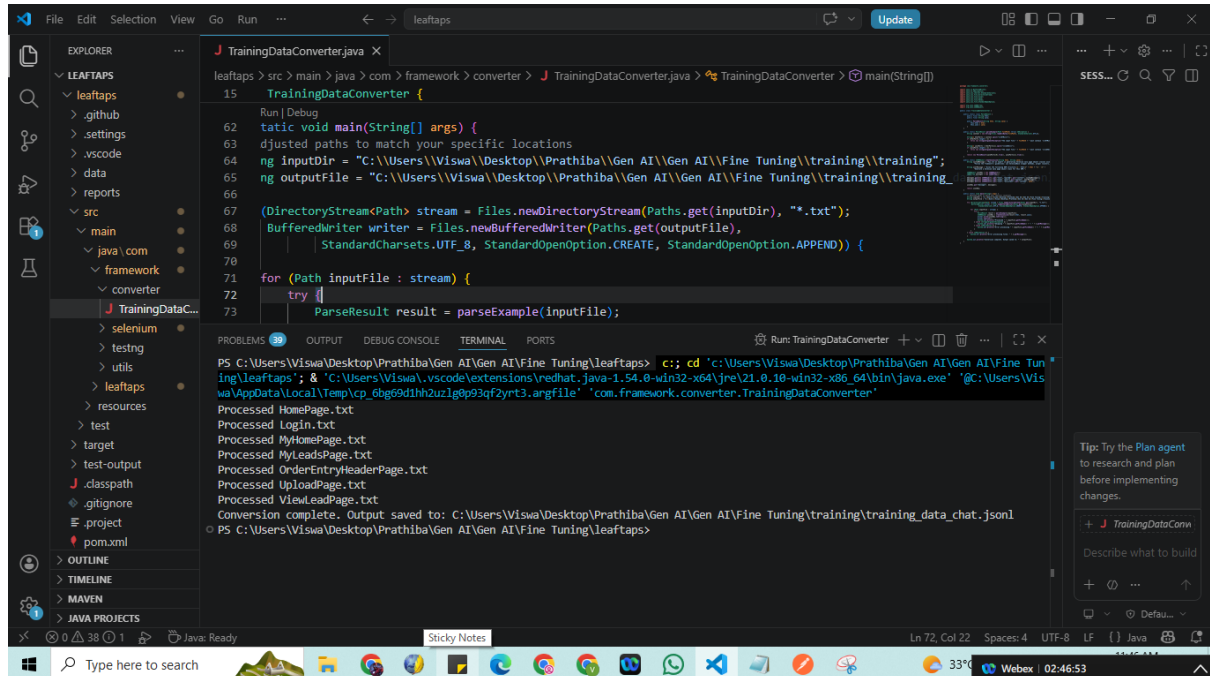


Week 4 Assignment : jsnol file preparation and code generation using finetned model in ai-extension.

Step 1 : Creating a training jsonl file



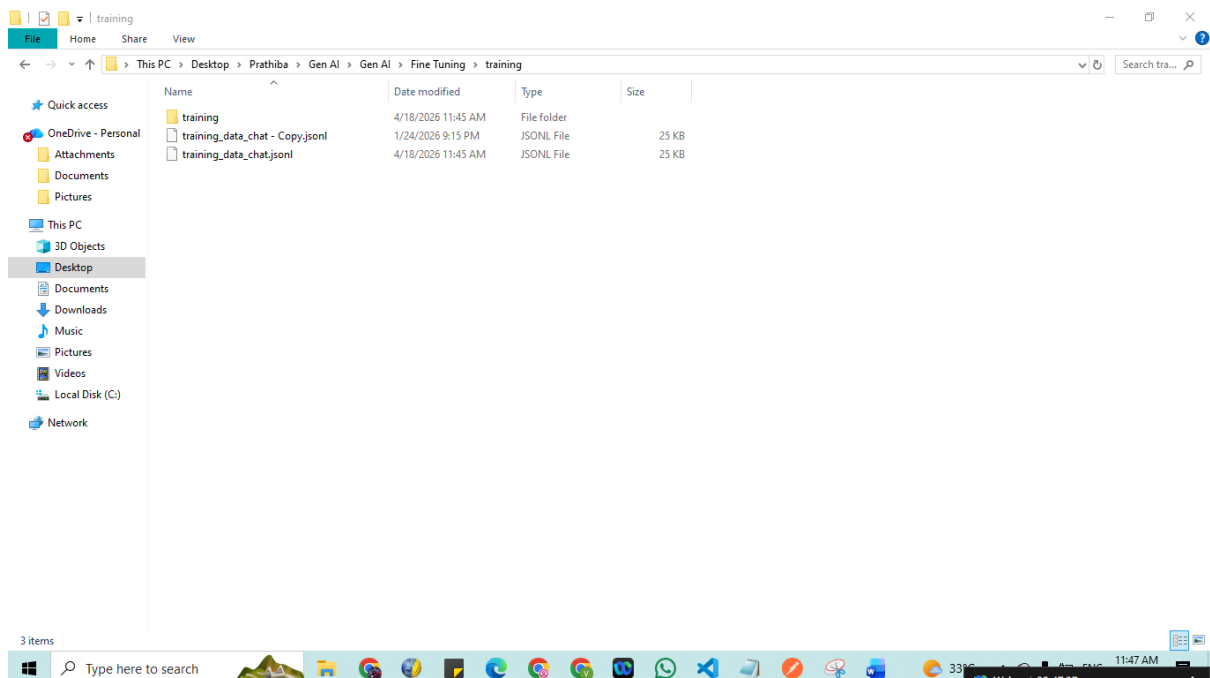
The screenshot shows the Visual Studio Code editor with the file `TrainingDataConverter.java` open. The code is a Java program that processes text files and converts them into a JSONL format. The output of the program is visible in the terminal window.

```
leafstaps > src > main > java > com > framework > converter > J TrainingDataConverter.java > TrainingDataConverter > main(String[])
15 TrainingDataConverter {
    Run | Debug
62 tatic void main(String[] args) {
63     djusted paths to match your specific locations
64     ng inputDir = "C:\\Users\\Viswa\\Desktop\\Prathiba\\Gen AI\\Fine Tuning\\training\\training";
65     ng outputFile = "C:\\Users\\Viswa\\Desktop\\Prathiba\\Gen AI\\Fine Tuning\\training\\training_
66
67     (DirectoryStream<Path> stream = Files.newDirectoryStream(Paths.get(inputDir), "*.txt");
68     BufferedWriter writer = Files.newBufferedWriter(Paths.get(outputFile),
69         StandardCharsets.UTF_8, StandardOpenOption.CREATE, StandardOpenOption.APPEND)) {
70
71     for (Path inputFile : stream) {
72         try {
73             ParseResult result = parseExample(inputFile);
74         } catch (IOException e) {
75             e.printStackTrace();
76         }
77     }
78 }
79 }
```

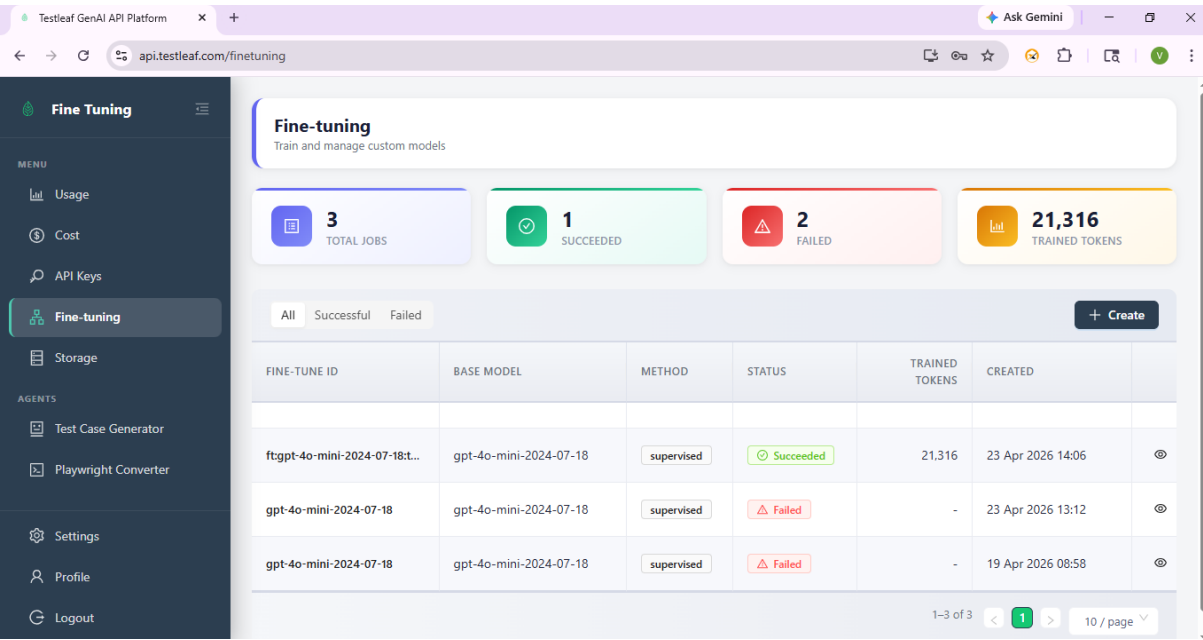
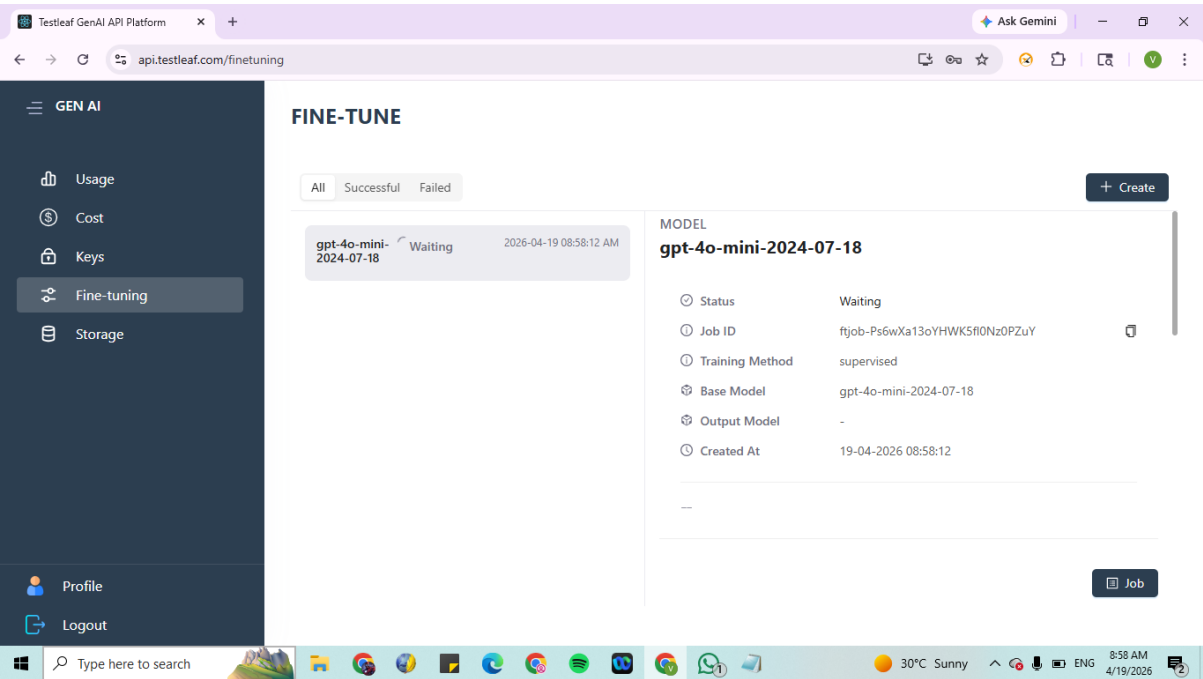
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Run: TrainingDataConverter

```
PS C:\Users\Viswa\Desktop\Prathiba\Gen AI\Fine Tuning\leafstaps> cd 'c:\Users\Viswa\Desktop\Prathiba\Gen AI\Fine Tuning\leafstaps'; & 'C:\Users\Viswa\AppData\Local\Microsoft\WindowsApps\java-1.54.0-win32-x64\jre\21.0.18-win32-x86_64\bin\java.exe' '@C:\Users\Viswa\AppData\Local\Temp\cp_6b6d1h2uzlg8p93qf2yrt3.argfile' 'com.framework.converter.TrainingDataConverter'
Processed HomePage.txt
Processed Login.txt
Processed MyHomePage.txt
Processed MyLeadsPage.txt
Processed OrderEntryHeaderPage.txt
Processed UploadPage.txt
Processed ViewLeadPage.txt
Conversion complete. Output saved to: C:\Users\Viswa\Desktop\Prathiba\Gen AI\Fine Tuning\training\training_data_chat.jsonl
PS C:\Users\Viswa\Desktop\Prathiba\Gen AI\Fine Tuning\leafstaps>
```



Step 2 : Creating a finetune llm



ft:gpt-4o-mini-2024-07-18:testleaf-2::DXjukGCl

✓ Status

Succeeded

🔍 Job ID

ftjob-XdKhE583dhIHKoHSXvga8Wlq

📘 Training Method

supervised

📦 Base Model

gpt-4o-mini-2024-07-18

📦 Output Model

ft:gpt-4o-mini-2024-07-18:testleaf-2::DXjukGCl

🕒 Created At

23 Apr 2026 14:06:07

📊 Trained Tokens

21,316

🔄 Epochs

4

📋 Batch Size

1

📈 LR Multiplier

0.16

🌟 Seed

1713910807

🔄 Re-run as Job

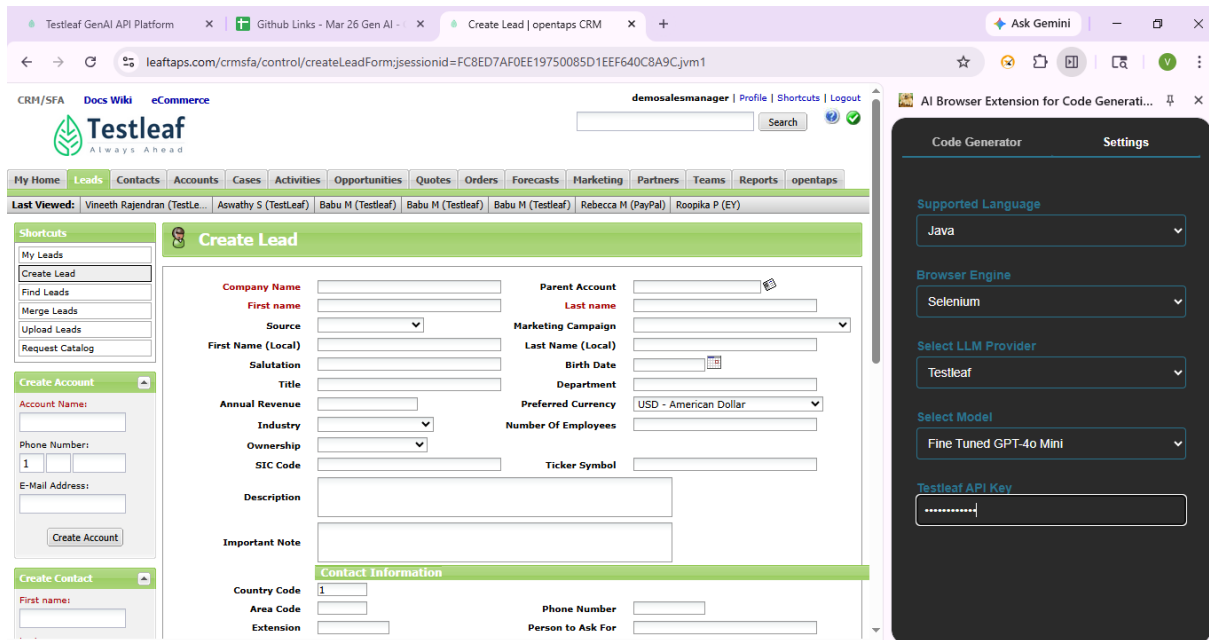
Close

Step 3 : Adding the fine-tuned Model into the Browser Extension

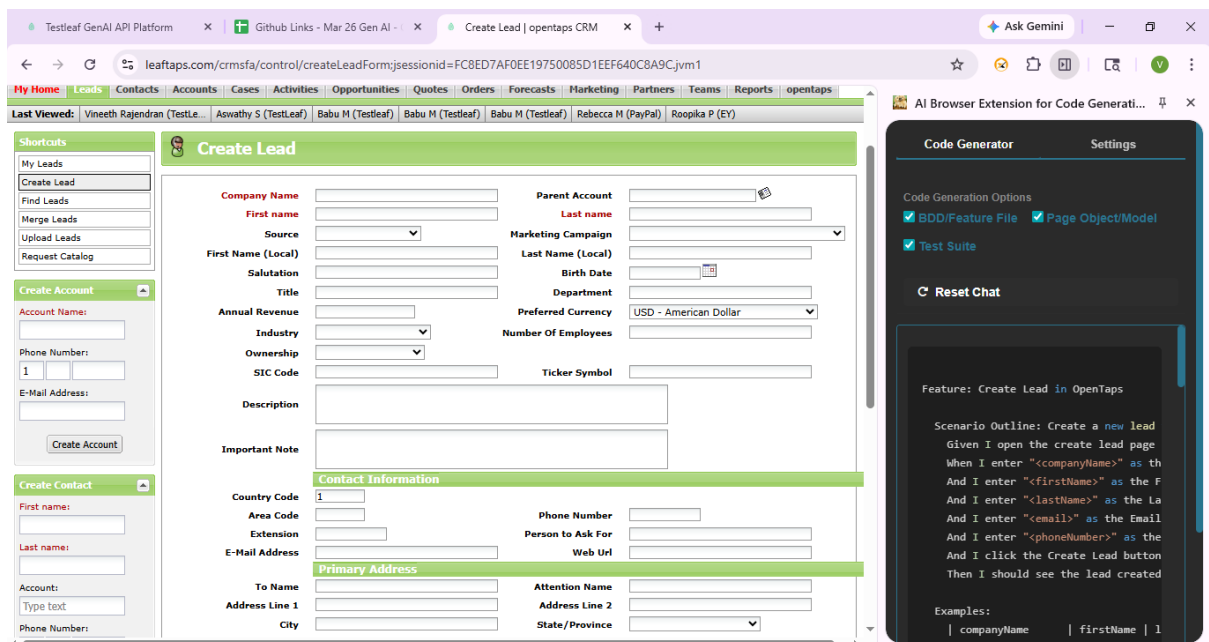
```
File Edit Selection View Go Run ... Browser Extension Update
EXPLORER
  BROWSER EXTENSION
    ai-extension
      assets
      lib
      src
        config
        content
        scripts
          api
            groq-api.js
            openai-api.js
            testleaf-api.js
            chat.js
            init-colors.js
            init-config.js
            popup.js
            prompts.js
            tabs.js
          styles
            chat.css
            sidepanel.css
            styles.css
            vars.css
          bg.js
          manifest.json
          panel.html
          ai-extension.zip
      OUTLINE
      TIMELINE
JS popup.js 1 X
ai-extension > src > scripts > JS popup.js > document.addEventListener('DOMContentLoaded') callback > modelsByProvider
5 document.addEventListener('DOMContentLoaded', async () => {
100
101 const providerSelect = document.getElementById('providerSelect');
102 const modelSelect = document.getElementById('modelSelect');
103 const groqApiKeyInput = document.getElementById('groqApiKey');
104 const openaiApiKeyInput = document.getElementById('openaiApiKey');
105 const testleafApiKeyInput = document.getElementById('testleafApiKey');
106
107 // Model options by provider
108 const modelsByProvider = {
109   groq: [
110     { value: 'deepseek-r1-distill-llama-70b', label: 'deepseek-r1-distill-llama-70b' },
111     { value: 'llama-3.3-70b-versatile', label: 'llama-3.3-70b-versatile' },
112     { value: 'openai/gpt-oss-120b', label: 'GPT-OSS-120B' }
113   ],
114   openai: [
115     { value: 'gpt-4o', label: 'GPT-4o' },
116     { value: 'gpt-4o-mini', label: 'GPT-4o Mini' },
117   ],
118   testleaf: [
119     { value: 'ft:gpt-4o-mini-2024-07-18:testleaf-2::DXjukGCl', label: 'Fine Tuned GPT-4o Mini' },
120   ]
121 };
122
123 // Function to update model options based on selected provider
124 function updateModelOptions(provider) {
125   modelSelect.innerHTML = '<option value="" disabled selected>Select the model...</option>';
126
127   if (provider) {
128     modelSelect.disabled = false;
129     const models = modelsByProvider[provider] || [];
130   }
131 }
```

Step 4 :

Finetune model is displayed in the Browser Extension



Step 5 : Selenium Code is generated with the fine tuned Model



Generated Code :

Feature: Create Lead in OpenTaps

Scenario Outline: Create a new lead with valid details

Given I open the create lead page

When I enter "<companyName>" as the Company Name

And I enter "<firstName>" as the First Name

And I enter "<lastName>" as the Last Name

And I enter "<email>" as the Email Address

And I enter "<phoneNumber>" as the Phone Number

And I click the Create Lead button

Then I should see the lead created successfully with "<companyName>"

Examples:

companyName	firstName	lastName	email	phoneNumber	
"Tata Consultancy"	"Rajesh"	"Kumar"	"rajesh@tcs.com"	"9876543210"	
"Infosys"	"Sita"	"Rani"	"sita@infosys.com"	"9876543211"	
"Wipro"	"Ravi"	"Sharma"	"ravi@wipro.com"	"9876543212"	

```
package com.leaf taps.stepdefs;
```

```
import io.cucumber.java.en.*;
```

```
import org.openqa.selenium.*;
```

```
import org.openqa.selenium.chrome.ChromeDriver;
```

```
import org.openqa.selenium.support.ui.*;
```

```
public class CreateLeadStepDefinitions {
```

```
    private WebDriver driver;
```

```
    private WebDriverWait wait;
```

```
@io.cucumber.java.Before
```

```
public void setUp() {
```

```
    System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");
```

```
    driver = new ChromeDriver();
```

```
    wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
    driver.manage().window().maximize();
```

```
}
```

```

@io.cucumber.java.After

public void tearDown() {
    if (driver != null) driver.quit();
}

@Given("I open the create lead page")
public void openCreateLeadPage() {

driver.get("https://leaftaps.com/crmsfa/control/createLeadForm;jsessionid=FC8ED7AF0EE19750085D1EEF640C8A9C.jvm1");

}

@When("I enter {string} as the Company Name")
public void enterCompanyName(String companyName) {
    WebElement companyNameField =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("createLeadForm_companyName")));
    companyNameField.sendKeys(companyName);
}

@When("I enter {string} as the First Name")
public void enterFirstName(String firstName) {
    WebElement firstNameField =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("createLeadForm_firstName")))
;
    firstNameField.sendKeys(firstName);
}

@When("I enter {string} as the Last Name")
public void enterLastName(String lastName) {
    WebElement lastNameField =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("createLeadForm_lastName")))
;

```

```

        lastNameField.sendKeys(lastName);
    }

    @When("I enter {string} as the Email Address")
    public void enterEmail(String email) {
        WebElement emailField =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("createLeadForm_primaryEmail")));
        emailField.sendKeys(email);
    }

    @When("I enter {string} as the Phone Number")
    public void enterPhoneNumber(String phoneNumber) {
        WebElement phoneField =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("createLeadForm_primaryPhoneNumber")));
        phoneField.sendKeys(phoneNumber);
    }

    @When("I click the Create Lead button")
    public void clickCreateLead() {
        WebElement createLeadButton =
wait.until(ExpectedConditions.elementToBeClickable(By.className("smallSubmit")));
        createLeadButton.click();
    }

    @Then("I should see the lead created successfully with {string}")
    public void verifyLeadCreation(String companyName) {
        WebElement leadConfirmation =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("viewLead_companyName_sp")));
        String createdCompanyName = leadConfirmation.getText();

        assert createdCompanyName.contains(companyName) : "Lead creation failed. Expected company name: " + companyName + " but found: " + createdCompanyName;
    }

```

```
}  
  
}
```

Playwright / Type Script Code generation

The screenshot shows a web browser with two tabs: 'Testleaf GenAI API Platform' and 'Create Lead | opentaps CRM'. The main window displays the 'Create Lead' form in the leafTaps CRM. The form is divided into several sections: 'Company Name' (First name, Last name, Source, Marketing Campaign), 'First Name (Local)', 'Last Name (Local)', 'Salutation', 'Birth Date', 'Title', 'Department', 'Annual Revenue', 'Preferred Currency', 'Industry', 'Ownership', 'Number Of Employees', 'SIC Code', 'Ticker Symbol', 'Description', and 'Important Note'. Below these are 'Contact Information' (Country Code, Area Code, Extension, Phone Number, Person to Ask For, E-Mail Address, Web Url) and 'Primary Address' (To Name, Attention Name, Address Line 1, Address Line 2, City, State/Province). On the left, there are 'Shortcuts' (My Leads, Create Lead, Find Leads, Merge Leads, Upload Leads, Request Catalog) and 'Create Account' (Account Name, Phone Number, E-Mail Address, Create Account) and 'Create Contact' (First name, Last name, Account, Type text, Phone Number, Create Account) forms. On the right, the 'AI Browser Extension for Code Generation' is open, showing 'Code Generator' and 'Settings' tabs. The 'Code Generator' tab has 'Supported Language' set to 'TypeScript', 'Browser Engine' set to 'Playwright', 'Select LLM Provider' set to 'Testleaf', and 'Select Model' set to 'Fine Tuned GPT-4o Mini'. The 'Testleaf API Key' is masked with asterisks.

This screenshot is similar to the previous one, but the 'AI Browser Extension for Code Generation' is now showing the generated TypeScript code in the 'Code Generator' tab. The 'Settings' tab remains the same. The generated code is as follows:

```
import { Page, Locator } from '@playwright/test';  
  
/**  
 * Page Object Model for Create Lead Form  
 */  
export class CreateLeadPage {  
  readonly page: Page;  
  readonly companyNameInput: Locator;  
  readonly firstNameInput: Locator;  
  readonly lastNameInput: Locator;  
  readonly submitButton: Locator;  
  
  constructor(page: Page) {  
    this.page = page;  
  }  
}
```

Generated Code :

```
import { Page, Locator } from '@playwright/test';
```

```
/**
```

```
 * Page Object Model for Create Lead Form
```

```

*/

export class CreateLeadPage {

    readonly page: Page;

    readonly companyNameInput: Locator;

    readonly firstNameInput: Locator;

    readonly lastNameInput: Locator;

    readonly submitButton: Locator;

    constructor(page: Page) {

        this.page = page;

        this.companyNameInput = page.locator('#createLeadForm_companyName');

        this.firstNameInput = page.locator('#createLeadForm_firstName');

        this.lastNameInput = page.locator('#createLeadForm_lastName');

        this.submitButton = page.locator('input[name="submitButton"]');

    }

    /**
     * Navigate to the Create Lead page
     */
    async goto() {

        await this.page.goto('/crmsfa/control/createLead');

    }

    /**
     * Fill in the company name
     * @param companyName - The name of the company
     */
    async fillCompanyName(companyName: string) {

        await this.companyNameInput.fill(companyName);

    }

```

```
/**
 * Fill in the first name
 * @param firstName - The first name of the lead
 */
async fillFirstName(firstName: string) {
  await this.firstNameInput.fill(firstName);
}
```

```
/**
 * Fill in the last name
 * @param lastName - The last name of the lead
 */
async fillLastName(lastName: string) {
  await this.lastNameInput.fill(lastName);
}
```

```
/**
 * Submit the create lead form
 */
async submitForm() {
  await this.submitButton.click();
}
}
```

```
import { test, expect } from '@playwright/test';
```

```
test.describe('Create Lead Form', () => {
  let page;
```

```
test.beforeEach(async ({ page: testPage }) => {
  page = testPage;
```

```
    await
page.goto('https://leaftaps.com/crmsfa/control/createLeadForm;jsessionid=FC8ED7AF0EE197
50085D1EEF640C8A9C.jvm1');

});
```

```
test.afterEach(async () => {

    // Optional: Add any cleanup code if necessary

});
```

```
test('should create a lead with valid data', async () => {

    await page.fill('#createLeadForm_companyName', 'Tech Solutions');
    await page.fill('#createLeadForm_firstName', 'Rajesh');
    await page.fill('#createLeadForm_lastName', 'Kumar');
    await page.selectOption('#createLeadForm_dataSourceId', 'LEAD_COLDSCALL');
    await page.selectOption('#createLeadForm_marketingCampaignId', '9002');
    await page.fill('#createLeadForm_primaryEmail', 'rajesh.kumar@example.com');
    await page.fill('#createLeadForm_primaryPhoneAreaCode', '91');
    await page.fill('#createLeadForm_primaryPhoneNumber', '9876543210');
    await page.fill('#createLeadForm_generalCity', 'Chennai');
    await page.fill('#createLeadForm_generalPostalCode', '600001');
    await page.selectOption('#createLeadForm_generalCountryGeold', 'IND');
    await page.click('input[type="submit"]');

    await expect(page).toHaveURL(/viewLead/);
    await expect(page.locator('h1')).toContainText('View Lead');

});
```

```
test('should show error for missing required fields', async () => {

    await page.click('input[type="submit"]');

    const errorMessages = await page.locator('.errorMessage').allTextContents();
    expect(errorMessages).toContain('Company Name is required');
```

```
expect(errorMessages).toContain('First name is required');
expect(errorMessages).toContain('Last name is required');
});
```

```
test('should not create a lead with invalid email', async () => {
  await page.fill('#createLeadForm_companyName', 'Tech Solutions');
  await page.fill('#createLeadForm_firstName', 'Rajesh');
  await page.fill('#createLeadForm_lastName', 'Kumar');
  await page.fill('#createLeadForm_primaryEmail', 'invalid-email');
  await page.click('input[type="submit"]');
```

```
  const errorMessage = await page.locator('.errorMessage').textContent();
  expect(errorMessage).toContain('Email address is invalid');
});
```

```
test('should create a lead with local names', async () => {
  await page.fill('#createLeadForm_companyName', 'Local Tech');
  await page.fill('#createLeadForm_firstName', 'Rajesh');
  await page.fill('#createLeadForm_lastName', 'Kumar');
  await page.fill('#createLeadForm_firstNameLocal', 'ரஈஜேஷ்');
  await page.fill('#createLeadForm_lastNameLocal', 'குமார்');
  await page.fill('#createLeadForm_primaryEmail', 'rajesh.kumar@example.com');
  await page.click('input[type="submit"]');
```

```
  await expect(page).toHaveURL(/viewLead/);
  await expect(page.locator('h1')).toContainText('View Lead');
});
});
```